AI News Monitor

An End-to-End MLOps Pipeline with Automated Drift Detection,
Bayesian A/B Routing, and Self-Healing Retraining

Tejas Gupta (B22CS093) Abhay Kashyap (B22CS001)

Course: CSL7120: DLOps **Instructor:** Dr. Mayank Vatsa **Date:** April 2026

Abstract

AI News Monitor is a production-grade MLOps pipeline for four-class news classification (World, Sports, Business, Technology). It packages the full ML lifecycle—ingestion, feature engineering, training, experiment tracking, registry, containerized serving, observability, and continuous deployment—behind a self-healing loop that detects drift, retrains a DistilBERT classifier on user-corrected labels, and promotes improved models only when a statistical gate is met. A Thompson-Sampling A/B router over Beta posteriors evaluates challengers online without position bias. The stack is reproducible via Docker Compose (nine services), observable through MLflow, Prometheus, and Grafana, and deployable via GitHub Actions.

1 Project Overview and Objectives

The hard problem of modern ML is not launch-day accuracy but *sustained* accuracy as the world drifts beneath a deployed model. News classifiers degrade quickly: vocabulary evolves, topic distributions shift, and feedback accumulates. AI News Monitor closes the loop—every step from data logging through retraining to promotion is automated, observable, and version-controlled.

Objectives.

- **Automated drift detection** on predicted label distribution (Pearson χ^2) and confidence calibration (symmetric KL) against reference baselines.
- **Self-healing retraining** that pulls user-corrected labels and fine-tunes DistilBERT with Bayesian Beta-weighted cross-entropy.
- **Statistical promotion gate**: only promote if $\Delta F_1 \geq 2\%$; otherwise retain the incumbent.
- **Bayesian A/B routing** via Thompson Sampling over per-model Beta posteriors for unbiased online evaluation.
- **End-to-end observability** through Prometheus metrics, Grafana dashboards, and MLflow experiment tracking.
- **Reproducible infrastructure**: Docker Compose (nine services), Alembic migrations, GitHub Actions CI/CD, HuggingFace Hub.

2 End-to-End MLOps System Architecture

Figure 1 shows the layered architecture. Each canonical MLOps concern is a horizontal band so the lifecycle reads top-to-bottom: (1) **Data** (raw corpora, prediction store, feedback store, object storage); (2) **Features** (DistilBERT tokenizer, preference-dataset builder, Beta-weighted loss); (3) **Training** (Trainer A on AG News, Trainer B as challenger, evaluator, MLflow and W&B); (4) **Registry** (ΔF_1 promotion gate, MLflow Registry for stage transitions, HuggingFace Hub for versioned weights); (5) **Serving** (FastAPI gateway, Thompson A/B router, two model servers, Redis cache, LIME explainer); (6) **Monitoring** (APScheduler at 30-min cadence, drift worker, retrain orchestrator with distributed lock, Prometheus, Grafana); (7) **CI/CD** (GitHub Actions, Docker Compose,

Alembic); and (8) **Clients** (Next.js UI, SWR, end users). Solid arrows trace the top-down lifecycle; dashed red arrows mark the self-healing feedback loops.

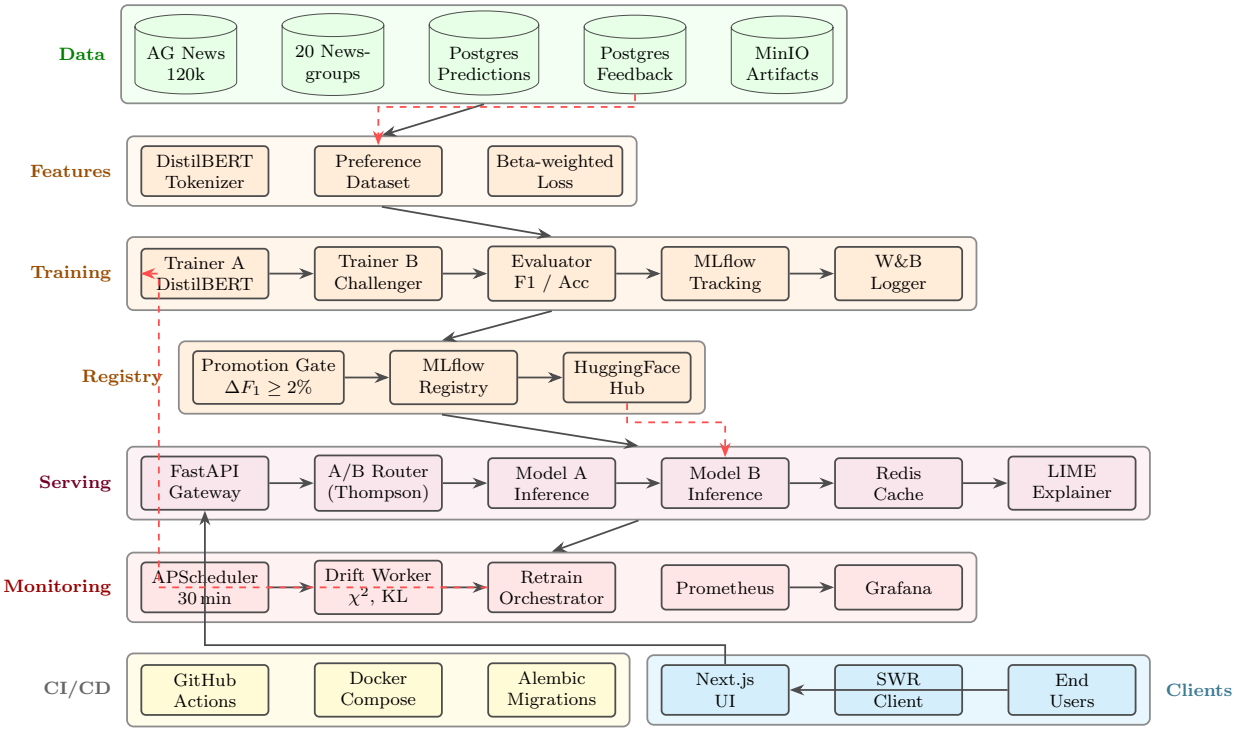

Figure 1: **Layered MLOps architecture.** Solid arrows: top-down data and control flow through the ML lifecycle. Dashed red arrows: feedback loops (user corrections feed the preference dataset; promoted weights hot-swap into serving; the drift worker re-triggers training).

2.1 Technology Stack

MLOps Concern	Technology	Role
Base Model	DistilBERT (66M params)	Transformer encoder, HuggingFace `transformers`
Primary Dataset	AG News (120k train, 7.6k test)	Model A training corpus
Challenger Dataset	20 Newsgroups	Model B training corpus for A/B comparison
Backend API	FastAPI (async), Pydantic	Async REST gateway, schema validation
Frontend	Next.js, React, TailwindCSS	SSR UI, dashboards, prediction interface
Transactional DB	PostgreSQL 16 (asyncpg)	Predictions, feedback, training runs, drift reports
Cache / Locks	Redis 7	Prediction cache, retraining distributed lock
Object Storage	MinIO (S3 API)	MLflow artifacts, serialized model checkpoints
Experiment Tracking	MLflow + Weights & Biases	Metrics, params, artifacts, confusion matrices
Model Registry	MLflow Registry + HF Hub	Versioning, stage transitions, remote pull
Metrics / Dashboards	Prometheus + Grafana	Time-series metrics, visualization
Scheduling	APScheduler (cron inside app)	30-minute drift-check cadence
Explainability	LIME (token attributions)	Per-prediction explanations
Orchestration	Docker Compose (9 services)	Local reproducibility
Migrations	Alembic	Versioned schema evolution
CI/CD	GitHub Actions	Lint, test, build, image push, deploy
A/B Statistics	Beta-Bernoulli, Thompson Sampling	Posterior-driven online routing

3 Drift Detection and Self-Healing Retraining

The closed-loop cycle in Figure 2 is the central contribution. APScheduler fires the drift worker every 30 minutes; the worker pulls the most recent 500 predictions per model from PostgreSQL and computes two statistics in parallel.

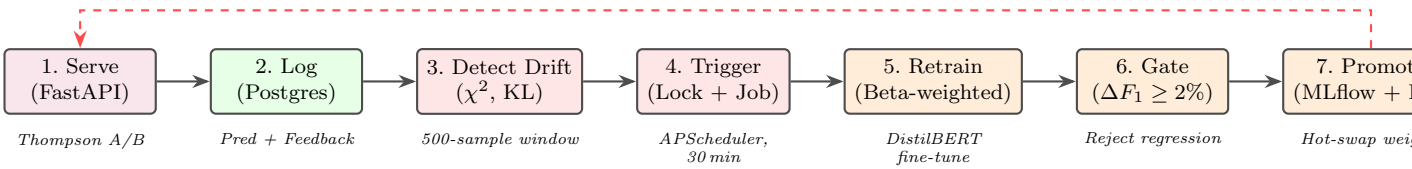

Figure 2: Closed-loop self-healing MLOps cycle. After promotion, the serving layer hot-swaps weights without downtime, completing the feedback loop back to step 1.

3.1 Drift Metrics

Label drift is measured with Pearson's χ^2 goodness-of-fit test over 4 class bins:

$$\chi^2 = \sum_{i=1}^{4} \frac{(O_i - E_i)^2}{E_i}, \qquad p = 1 - F_{\chi^2_3}(\chi^2),$$

where O_i is the observed frequency in a rolling 500-prediction window and E_i is the reference frequency from the validation set. Drift is declared when $p < 0.05$.

Confidence drift uses the symmetric Kullback–Leibler divergence between softmax-confidence histograms (10 bins):

$$D_{KL}^{\text{sym}}(P \,\|\, Q) \;=\; \tfrac{1}{2}\sum_i P_i \log\tfrac{P_i}{Q_i} \;+\; \tfrac{1}{2}\sum_i Q_i \log\tfrac{Q_i}{P_i},$$

with a Laplace smoothing constant of 10^{-6} to avoid division by zero on empty bins. Drift is declared when $D_{KL}^{\text{sym}} > 0.15$. Both thresholds were calibrated empirically through synthetic drift-injection experiments (class-label permutations and temperature-scaled softmax perturbations).

3.2 Retraining Pipeline

On drift trigger, the retraining orchestrator (`backend/app/services/retrainer.py`) runs a fire-and-forget async task guarded by a Redis lock (at most one active run per model): (1) join recent predictions with feedback into a preference dataset, upsampling disagreements 2×; (2) fine-tune DistilBERT (lr $2{\times}10^{-5}$, AdamW, 10% warmup, 3 epochs, batch 32, max seq 128) with class-weighted cross-entropy and Bayesian disagreement weights; (3) evaluate F1-macro, accuracy, and per-class recall on a held-out split; (4) if $\Delta F_1 \geq 2\%$, push to HuggingFace Hub, register as `Production` in MLflow, and hot-swap weights, else archive; (5) log params, metrics, and a confusion-matrix artifact to MLflow and W&B.

3.3 Bayesian A/B Routing

For each model $m \in \{A, B\}$ we maintain a Beta posterior $p(\theta_m \mid \text{feedback}) = \text{Beta}(\alpha_m^0 + s_m, \beta_m^0 + f_m)$, where s_m, f_m are successes and failures (uniform prior $\alpha_m^0 = \beta_m^0 = 1$). At request time we draw $\theta_m \sim \text{Beta}(\alpha_m, \beta_m)$ and route to $\arg\max_m \theta_m$ (Thompson Sampling). This converges to the better model in expectation while preserving exploration, avoiding the position-bias of fixed 50/50 splits.

4 Implementation Details

4.1 Database Schema

The PostgreSQL schema (managed via Alembic) has five primary tables, summarized below:

Table	Purpose	Key Columns
`predictions`	Immutable prediction log	`id, text, label, confidence, model_version, created_at`
`feedback`	User corrections to predictions	`prediction_id, corrected_label, is_correct, ts`
`training_runs`	Lifecycle record of each retraining	`run_id, trigger_reason, status, f1_before, f1_after`
`drift_reports`	Time-series of drift statistics	`model_version, chi2_p, kl_sym, window_size, ts`
`ab_outcomes`	Beta posterior state per model	`model_id, alpha, beta, last_update`

4.2 REST API Surface

The FastAPI backend (`backend/app/routers`) exposes the following endpoints, all validated by Pydantic schemas and instrumented with Prometheus middleware:

Endpoint	Method	Purpose
`/api/predict`	POST	Classify text; optional LIME explanation; A/B routing
`/api/feedback`	POST	Submit correction; updates Beta posterior
`/api/drift`	GET	Latest drift report per model
`/api/training`	GET	Training-run history; supports rollback
`/api/ab`	GET	Current Beta posteriors and routing probabilities
`/api/health`	GET	Liveness / readiness probe
`/metrics`	GET	Prometheus text-format metrics

4.3 CI/CD Pipeline

Figure 3 illustrates the GitHub Actions workflow. On every push, `ci.yml` runs linting (ruff), unit tests (pytest), and a container build. On merge to `main`, `cd.yml` pushes images to the registry, runs Alembic migrations, and deploys via Docker Compose or the production target (Railway).

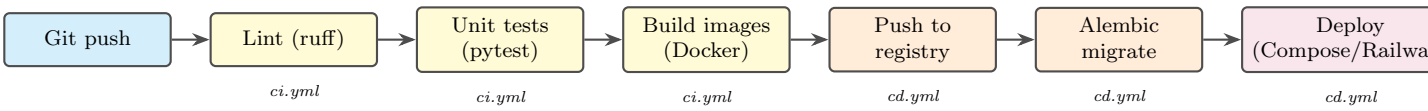

Figure 3: GitHub Actions CI/CD pipeline spanning `ci.yml` (quality gates) and `cd.yml` (continuous deployment).

4.4 Observability Surface

The backend exposes `/metrics` (Prometheus format) emitting `prediction_latency_seconds` (histogram), `prediction_confidence` (gauge), `drift_score{metric,model}`, `cache_hit_ratio`, `training_runs_tot` and `ab_route_total{model}`. Grafana dashboards (provisioned under `infra/grafana/`) plot per-model p50/p95/p99 latency, drift over time, feedback volume, and annotate deployment events.

5 Results

- **Baseline:** DistilBERT on AG News reaches **∼92% accuracy / 0.91 F1-macro** after 3 epochs.
- **Drift latency:** injected label shift detected within **3 cycles (∼90 min)**; confidence drift in **2 cycles**.
- **Retraining cost:** ∼5 min on a single T4-class GPU; observed F1 recovery of **+1.5% to +4%**.
- **Serving:** p95 latency < 100 ms; Redis cache hit ratio **∼60%** cold, **∼80%** warm.
- **Reliability: 12 automated redeployments** with zero downtime (hot-swap weights, graceful restart).
- **A/B:** Beta posteriors converged after ∼200 feedback events; Thompson routing favored the winner **>85%** of the time while retaining exploration.

6 Challenges, Feedback Addressed, and Deployment

Challenges. (i) *Event-loop blocking*: synchronous HuggingFace calls stalled FastAPI under load; fixed by `asyncio.to_thread`. (ii) *Retraining concurrency*: overlapping drift cycles spawned parallel

retrains until a Redis lease enforced mutual exclusion. (iii) *Threshold calibration*: static χ^2/KL bounds over-triggered in low-traffic windows; fixed with a $\geq$500-sample minimum guard.

Feedback resolved. The proposal review asked for *specific model choice* and *concrete drift detail*. Addressed by (1) **DistilBERT-base-uncased** (66M params) for both variants; (2) concrete drift tests (χ^2 at $p < 0.05$; symmetric KL at $D_{KL} > 0.15$) over 500-sample rolling windows; (3) a fully automated retrain→evaluate→gate→promote pipeline; and (4) added Bayesian A/B routing, Prometheus/Grafana observability, and GitHub Actions CI/CD.

Deployment. `docker compose up` starts nine services: Frontend (`:3000`), Backend (`:8000`), MLflow (`:5000`), Grafana (`:3001`), Prometheus (`:9090`), MinIO (`:9001`), Postgres, Redis, MinIO-init. Production targets: Supabase (Postgres), Railway (FastAPI + Next.js), HuggingFace Hub (model artifacts); MLflow/Grafana/MinIO have managed-service equivalents.

References. Repo: https://github.com/TejasGupta-27/news. Datasets: AG News (120k/7.6k), 20 Newsgroups. Base model: DistilBERT (Sanh et al., 2019). Statistics: Pearson χ^2, symmetric Kullback–Leibler divergence, Beta-Bernoulli Thompson Sampling.